

EE 448 TINYML LAB 8:

Name: Mihir Sharma

Github: <https://github.com/mihirscsharma/ee-446/tree/main/lab%208>

Edge Impulse: <https://studio.edgeimpulse.com/studio/1016120>

## Part 1

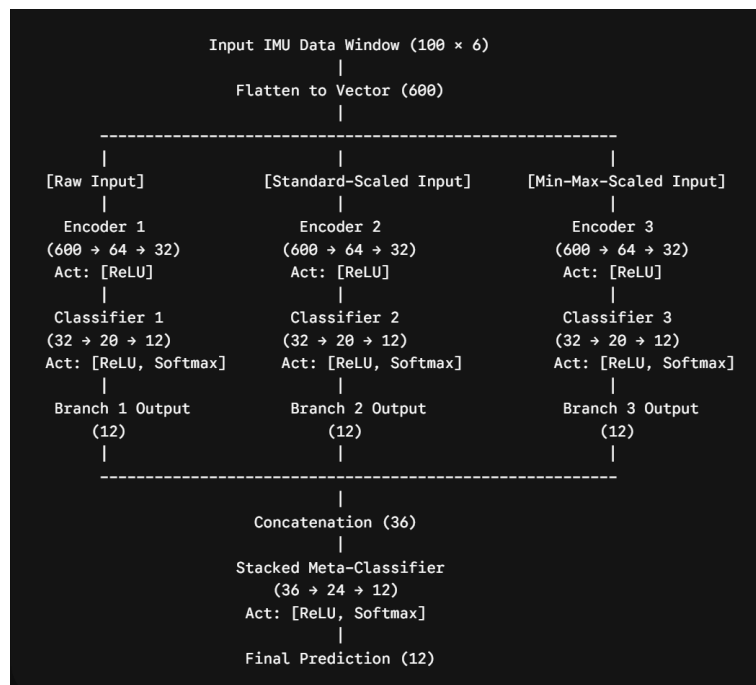
```
Timing: DSP 80 ms, inference 533 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 533 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  id: 0.011719
  lift: 0.988281
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 580 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id: 0.902344
  lift: 0.097656
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  id: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  id: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
```

```

Timing: DSP 84 ms, inference 568 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 540 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 541 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 552 us, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 551 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 551 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 551 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...

```

## Part 2



The ensemble model flattens a  $100 \times 6$  IMU data window into a 600-dimensional vector and

processes it in parallel across three branches: raw, standard-scaled, and min-max scaled. Each branch uses a ReLU-activated encoder ( $600 \rightarrow 64 \rightarrow 32$ ) and a softmax classifier ( $32 \rightarrow 20 \rightarrow 12$ ) to produce a 12-class probability vector. These three outputs are concatenated into a 36-dimensional vector and fed into a final stacked meta-classifier ( $36 \rightarrow 24 \rightarrow 12$ ) to generate the ultimate prediction. By combining three encoders, three classifiers, and one meta-classifier, this seven-model system leverages diverse data distributions to increase accuracy.

2. To optimize the model for resource-constrained TinyML microcontrollers, it undergoes a compression pipeline that begins with magnitude-based pruning, gradually increasing weight sparsity from 10% to 80% over 500 training steps. After stripping the pruning wrappers to eliminate structural overhead, the model undergoes Quantization Aware Training (QAT) to adapt to low-precision arithmetic, utilizing a custom callback to ensure pruned weights stay exactly at zero. Finally, a representative dataset is used to calibrate real activation ranges, allowing the model to be converted into a fully int8 TensorFlow Lite format. This combination of pruning and quantization drastically reduces the model's memory and computational requirements without sacrificing strong classification performance.

3.

```
-----  
Final prediction: Standing still  
Class index: 0  
Confidence score: 0.3867  
-----  
Samples collected: 25  
Samples collected: 50  
Samples collected: 75  
Samples collected: 100  
Window collected. Running ensemble inference...  
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0039, 0.0078, 0.0039, 0.0000, 0.0078, 0.0000, 0.0039, 0.0000, 0.9766  
Standard-scaled model scores: 0.0000, 0.0273, 0.0000, 0.0000, 0.3477, 0.0000, 0.0000, 0.6172, 0.0000, 0.0078, 0.0000, 0.0000  
Min-max-scaled model scores: 0.0000, 0.0000, 0.8164, 0.0000, 0.1836, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000  
Meta-classifier scores: 0.3984, 0.0430, 0.2148, 0.0117, 0.0820, 0.0078, 0.0195, 0.0039, 0.0078, 0.0117, 0.0078, 0.1914  
-----  
Final prediction: Standing still  
Class index: 0  
Confidence score: 0.3984  
-----  
Samples collected: 25  
Samples collected: 50  
Samples collected: 75  
Samples collected: 100  
Window collected. Running ensemble inference...  
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0039, 0.0078, 0.0039, 0.0000, 0.0078, 0.0000, 0.0039, 0.0000, 0.9766  
Standard-scaled model scores: 0.0000, 0.0273, 0.0000, 0.0000, 0.3750, 0.0000, 0.0000, 0.5898, 0.0000, 0.0078, 0.0000, 0.0000  
Min-max-scaled model scores: 0.0000, 0.0000, 0.7891, 0.0000, 0.2070, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000  
Meta-classifier scores: 0.3867, 0.0430, 0.1992, 0.0117, 0.1055, 0.0078, 0.0195, 0.0039, 0.0078, 0.0117, 0.0078, 0.1992  
-----  
Final prediction: Standing still  
Class index: 0  
Confidence score: 0.3867  
-----
```

The TinyML ensemble pipeline successfully executes real time inference on the Arduino, processing 100 sample IMU windows per cycle within the device hardware constraints. For the resting board condition, the overall output is acceptable. Despite internal disagreements among the submodels, the stacked meta classifier correctly and consistently generates a final prediction of "Standing still" (class 0), proving that the complete preprocessing and classification pipeline functions properly on the embedded device.

However, there are noticeable issues regarding model confidence and conflicting branch outputs. The raw data classifier produces an unexpected, highly confident prediction for class 11 (approximately 0.98 confidence), which heavily conflicts with the standard scaled and min max scaled branches. This discrepancy strongly suggests a baseline offset, such as sensor noise or an orientation mismatch between the live board on your desk and the original training data. Because the meta classifier must synthesize these conflicting inputs, the final ensemble confidence drops to a relatively low 0.39. While the prediction remained stable during testing, this narrow confidence margin leaves the system vulnerable to erratic jumps if the sensor noise changes slightly.

To improve deployment reliability, a couple of practical changes could be implemented. First, introducing temporal smoothing, such as a majority vote filter across three to five consecutive inference windows, would stabilize the output and prevent flickering between states. Second, adding a startup calibration routine when the board boots up to capture resting sensor values and calculate an offset would help align the live physical data with the feature distributions expected by the trained models, thereby reducing branch conflict and boosting final confidence.

4. Yes, during deployment, the ensemble model exhibited unexpected behavior by repeatedly predicting "Standing still" across multiple inference windows, regardless of how the board was actually moved. This issue primarily stems from a domain shift between the original mHealth training dataset and the live data collected by the Arduino Nano 33 BLE Sense. Several factors contribute to this discrepancy, including differences in sensor hardware such as accelerometer sensitivity and noise profiles, as well as mismatches in board orientation that alter the gravity vectors on the X, Y, and Z axes. Furthermore, slight deviations in real time preprocessing, standard scaling, or unit conversions can cause the live inputs to fall outside the ranges the model learned during training. Combined with the fact that live testing motions rarely perfectly replicate the exact movements of the original dataset participants, the model struggles to interpret the unfamiliar incoming data. Consequently, the ensemble defaults to a "safe" or familiar baseline class, resulting in reduced confidence, distinct class bias, and repetitive predictions.